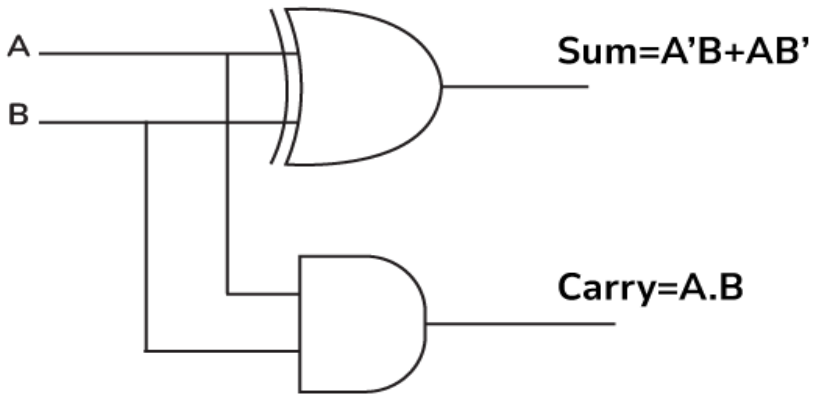
HDL Lab Experiments

Faculty: Dr. Jisha P

Half adder

Half Adder



Truth Table

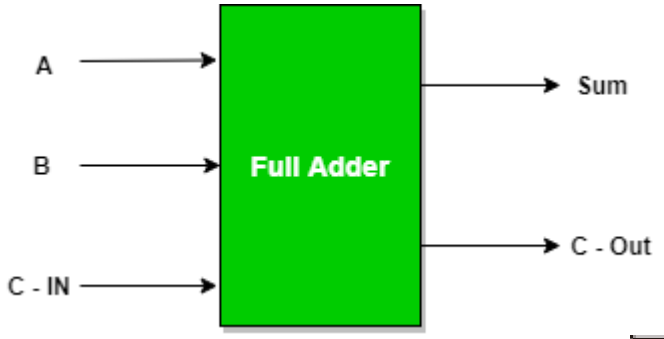
A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

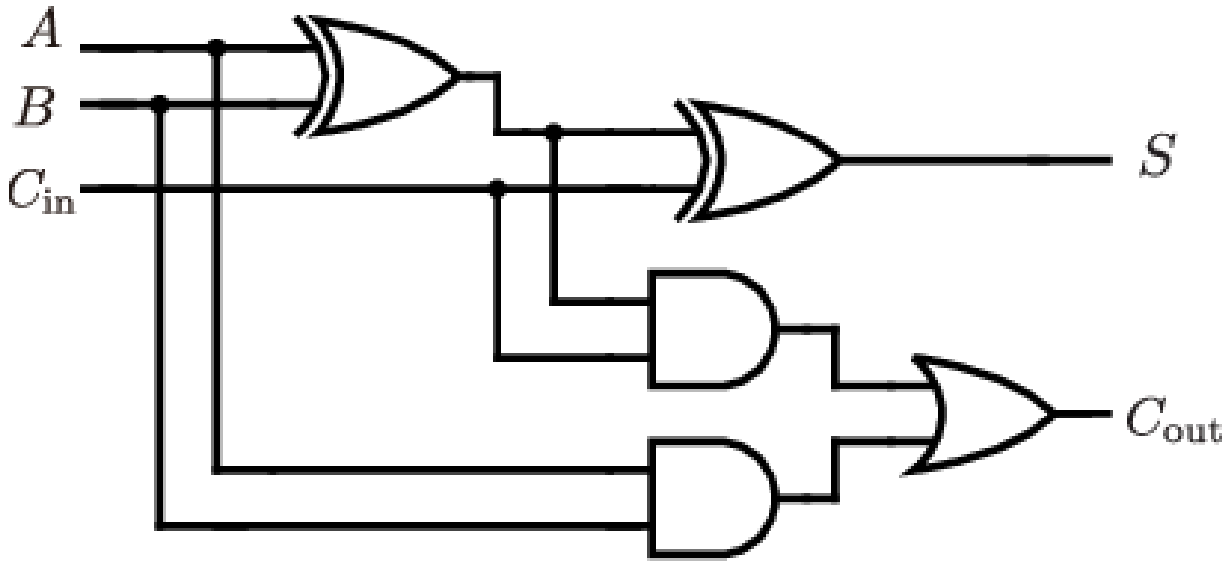
1.a- Gate-level modelling: Half adder

```
module half_adder (

input a,b,

output sum,carry);

xor (sum,a,b);

and (carry,a,b);

endmodule
```

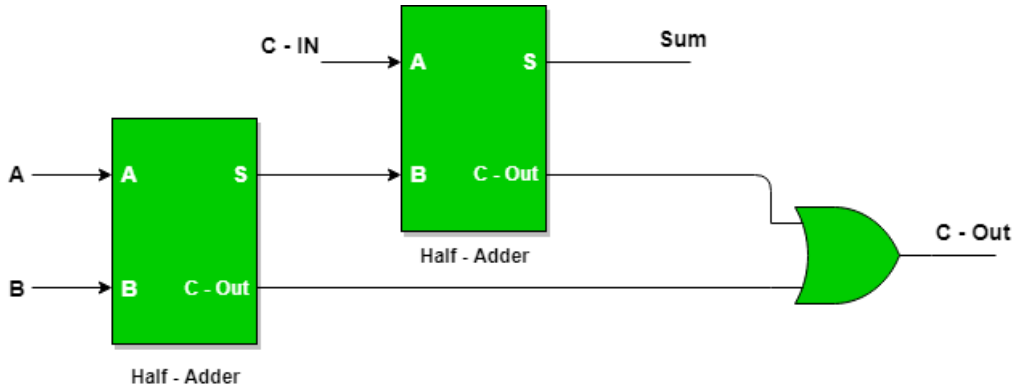
```
module half_adder_tb;
reg a,b;
wire sum,carry;
half_adder ha (a,b,sum,carry);
initial
begin
a=0;b=0;
#10 a=0;b=1;
#10 a=1;b=0;
#10 a=1;b=1;
#10
$finish;
end
endmodule
```

2. Full Adder





Inputs			Outputs	
A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



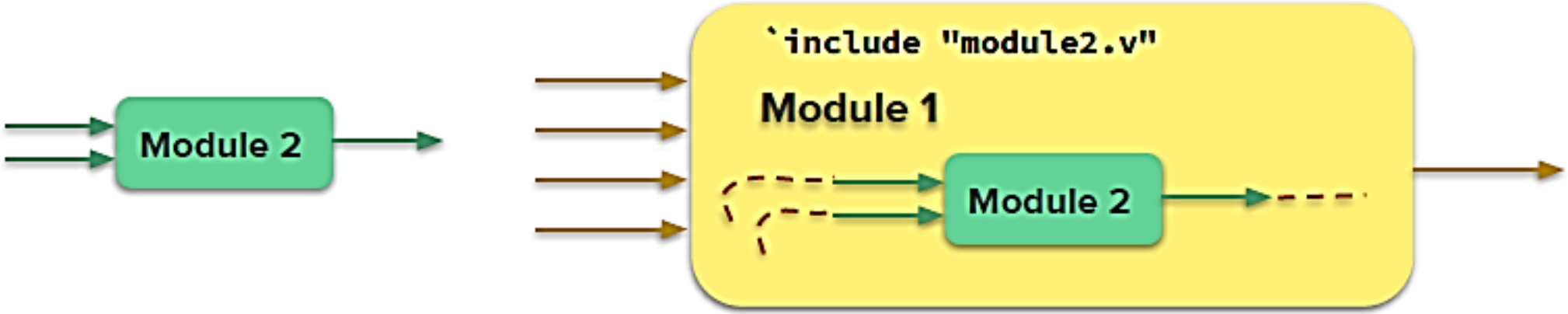
b. Gate-level modelling: Full adder

```
module full_adder (input a,b,c,
output sum,carry);
wire y1,y2,y3;
xor g1 (sum,a,b,c);
and g2 (y1,a,b);
and g3 (y2,b,c);
and g4 (y3,a,c);
or g5 (carry,y1,y2,y3);
endmodule
```

```
module full_adder_tb;
reg a,b,c;
wire sum,carry;
full_adder fa(a,b,c,sum,carry);
initial begin
a=0;b=0;c=0;
#10 a=0;b=0;c=1;
#10 a=0;b=1;c=0;
#10 a=0;b=0;c=1;
#10 a=1;b=0;c=0;
#10 a=1;b=0;c=1;
#10 a=1;b=1;c=0;
#10 a=1;b=1;c=1;
#10 $finish;
end
endmodule
```

Verilog Language Features

Two Ways to Specify Connectivity During Module Instantiation



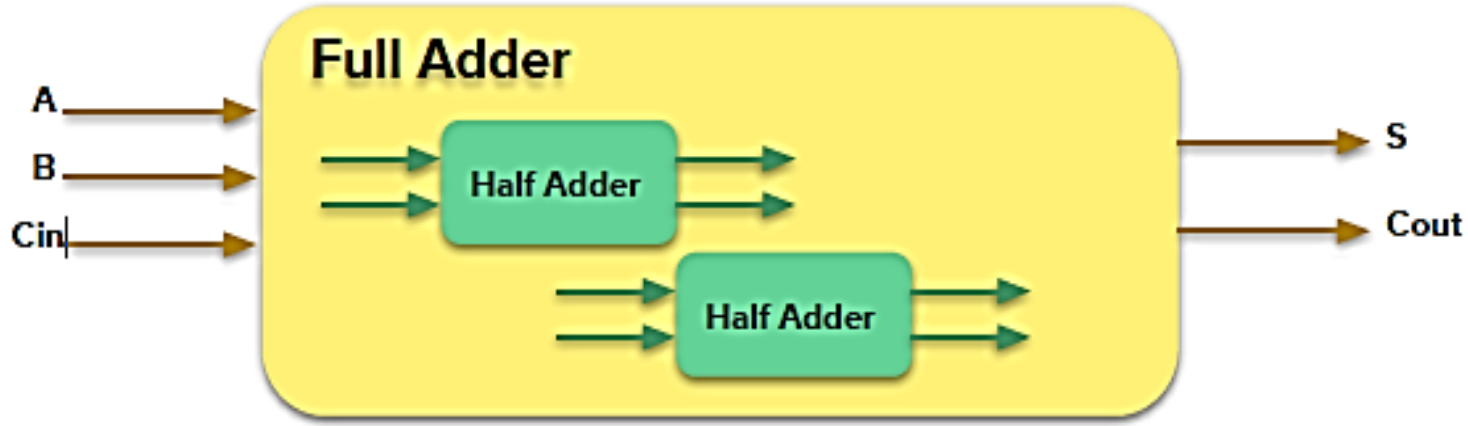
Connectivity of the signal lines between two modules can be specified in 2 ways:

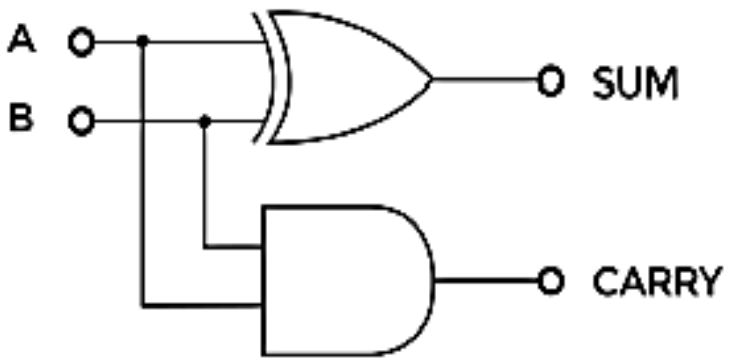
- **Positional Association (Implicit)**
  - Parameters listed **in the same order** as in the original module description.
- **Explicit Association**
  - Parameters **explicitly** listed **in arbitrary order**.

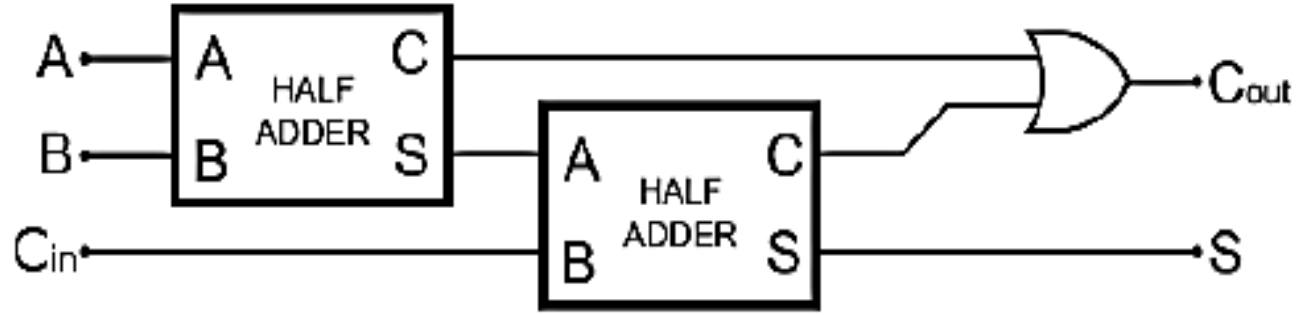
Verilog Language Features

- Positional Association Example - Full Adder Using Half Adder Module

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

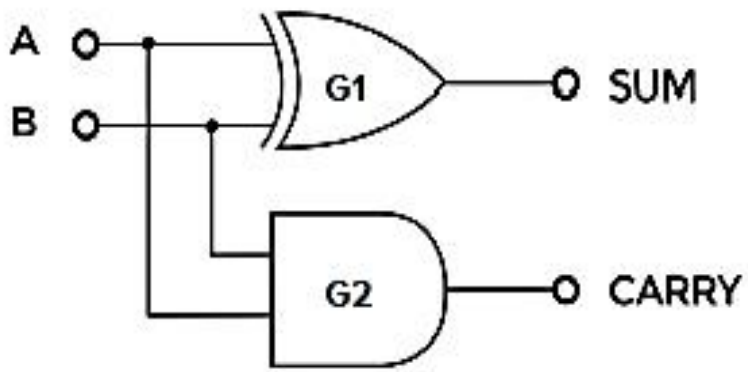






Verilog Language Features

- Positional Association Example - Full Adder Using Half Adder Module




```
module half_adder (Sum, Carry, A, B);
    input A, B;
    output Carry, Sum;
    //structural description
    xor G1(Sum, A, B);
    and G2(Carry, A, B);
endmodule
```

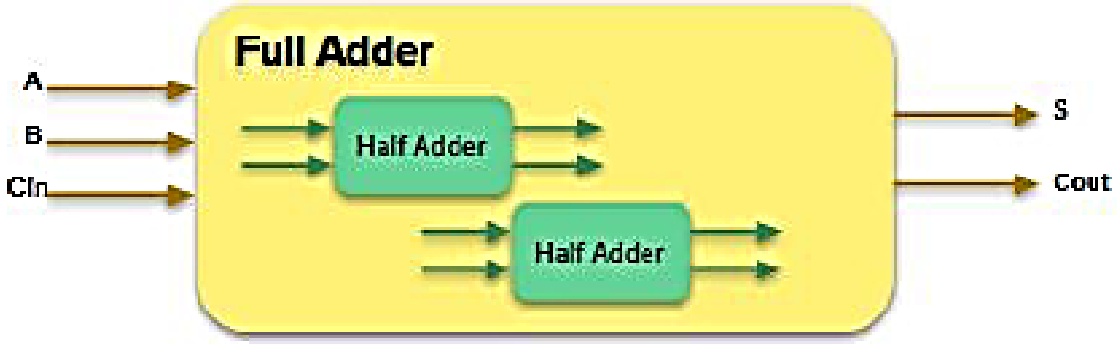
=

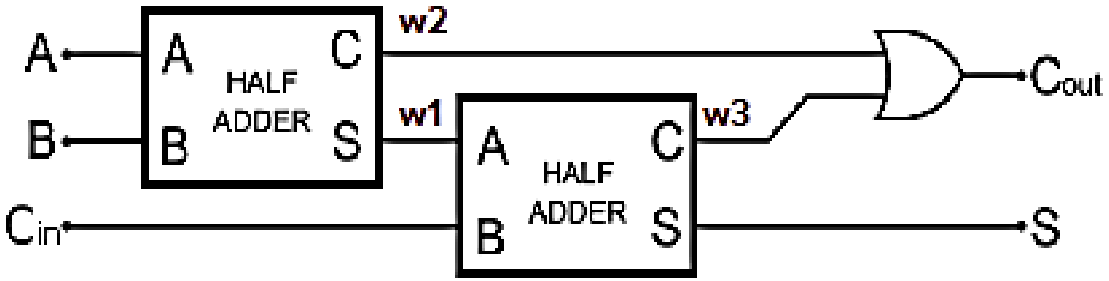
```
//behavioral description
assign Sum = A ^ B;
assign Carry = A & B;
```

Verilog Language Features

Note the port order

- Positional Association Example - Full Adder Using Half Adder Module



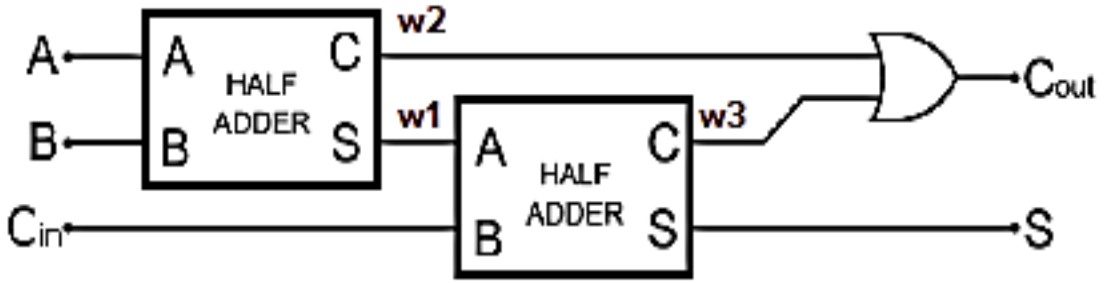



```
module half_adder (Sum, Carry, A, B);
```

```
module full_adder (Sum, Cout, A, B, Cin);
    input A, B, Cin;
    output Cout, Sum;
    wire w1, w2, w3;
    half_adder HA1 (w1, w2, A, B);
    half_adder HA2 (Sum, w3, Cin, w1);
    or (Cout, w2, w3);
endmodule
```

Verilog Language Features

- Explicit Association Example - Full Adder Using Half Adder Module



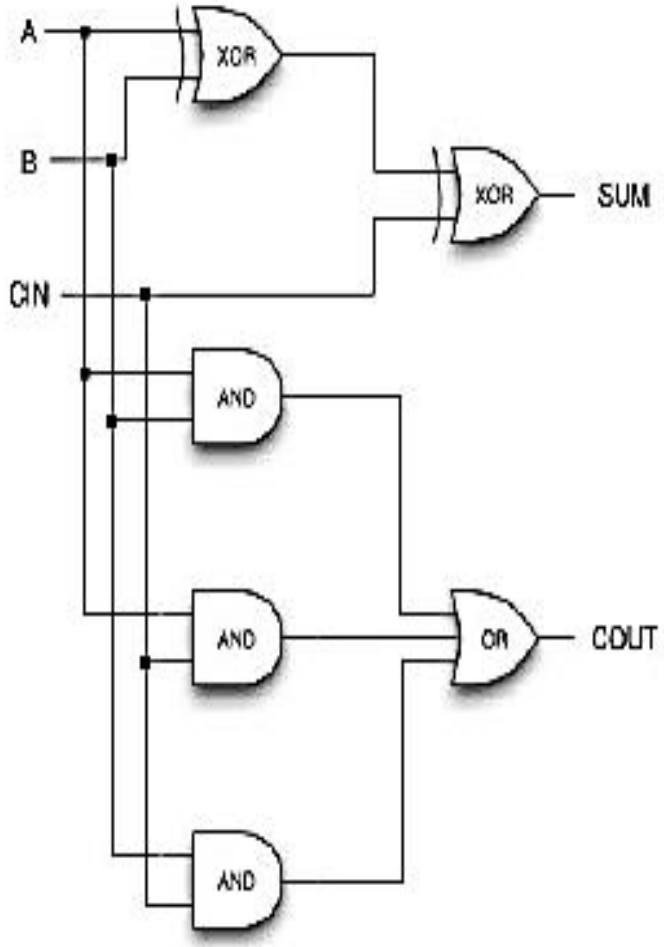

```
module half_adder (Sum, Carry, A, B);
```

Ports are explicitly specified - order is not important

```
module full_adder (Sum, Cout, A, B, Cin);
    input A, B, Cin;
    output Cout, Sum;
    wire w1, w2, w3;
    half_adder HA1 (.A(A), .B(B), .Sum(w1), .Carry(w2));
    half_adder HA2 (.Sum(Sum), .Carry(w3), .B(Cin), .A(w1));
    or (Cout, w2, w3);
endmodule
```

Less chance for errors

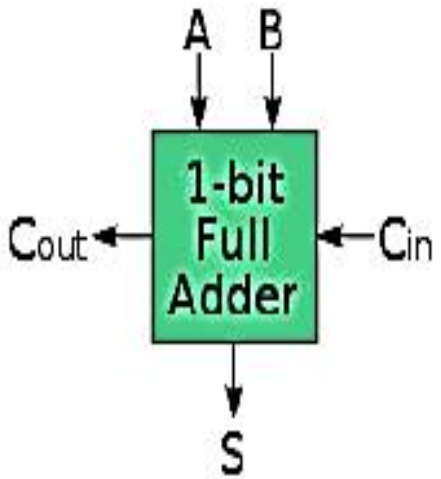
A	B	Cin	S	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1




```
sum = (A^B)^C
Cout = AB+BC+AC
```

Full_Adder.v module

```
module Full_Adder(
    input A,
    input B,
    input Cin,
    output S,
    output Cout
    );
    assign S = (A^B)^Cin;
    assign Cout = (A&B)|(B&Cin)|(Cin&A);
endmodule
```



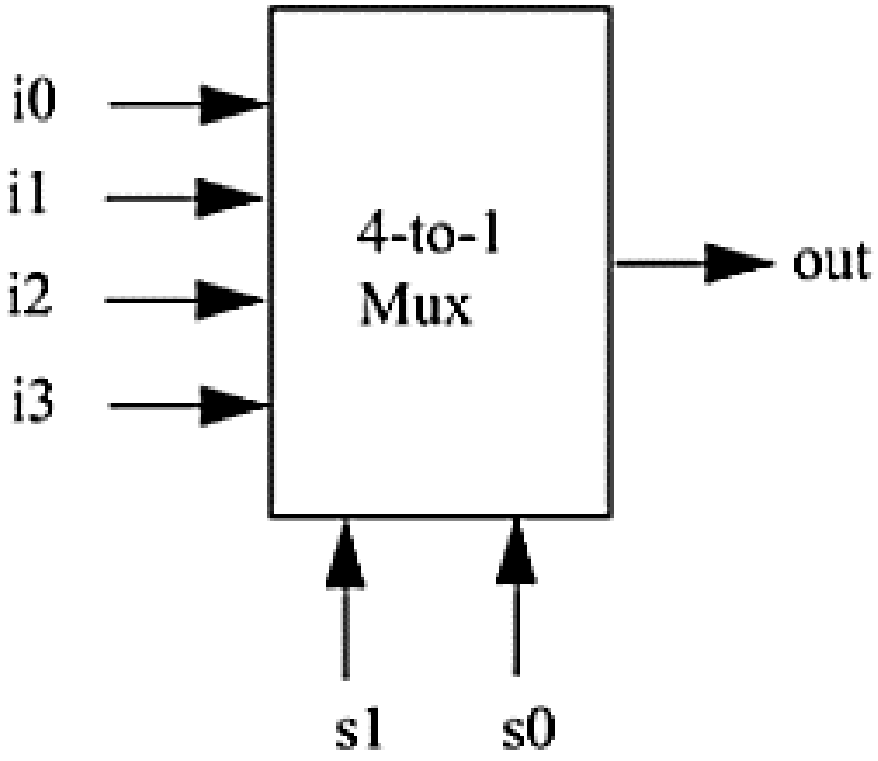
Note the behavioral description

Simulation results:

2. Gate-level modelling: **Multiplexers** and demultiplexers

https://prezi.com/view/6jxQNVPgPdGqST6i9Xzk/

4-to-1 Multiplexer I/O Diagram



Truth Table

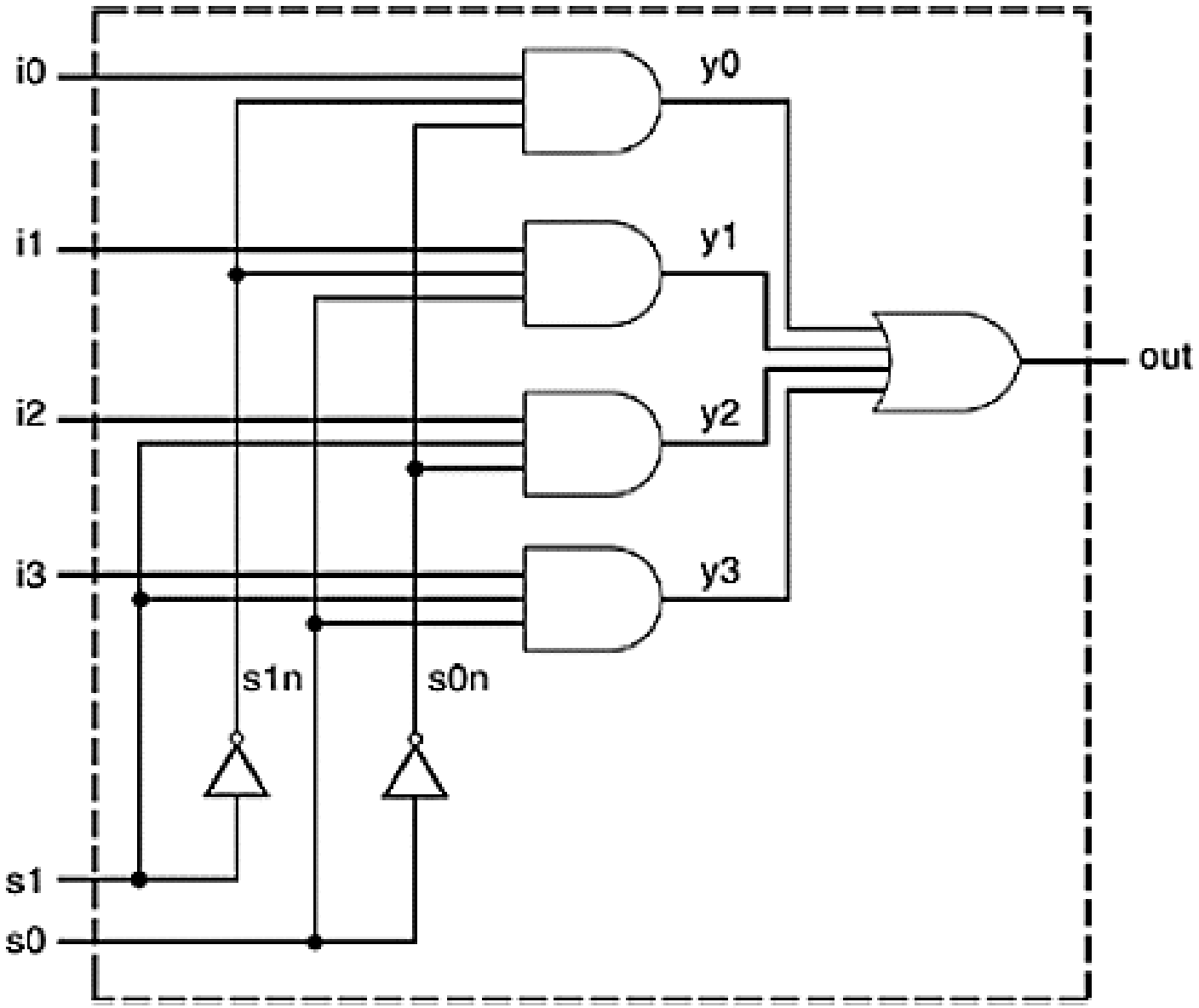
S0	S1	Y
0	0	I0
0	1	I1
1	0	I2
1	1	I3

So, final equation,

$Y = S0'.S1'.I0 + S0'.S1.I1 + S0.S1'.I2 + S0.S1.I3$

4-to-1 Multiplexer Logic Diagram

- The logic diagram has a one-to-one correspondence with the Verilog description.
- Two intermediate nets, **s0n and s1n**, are created; they are complements of input signals **s1 and s0**.
- Internal nets **y0, y1, y2, y3** are also required.
- Instance names are optional for Verilog primitives but are mandatory for instances of user-defined modules.



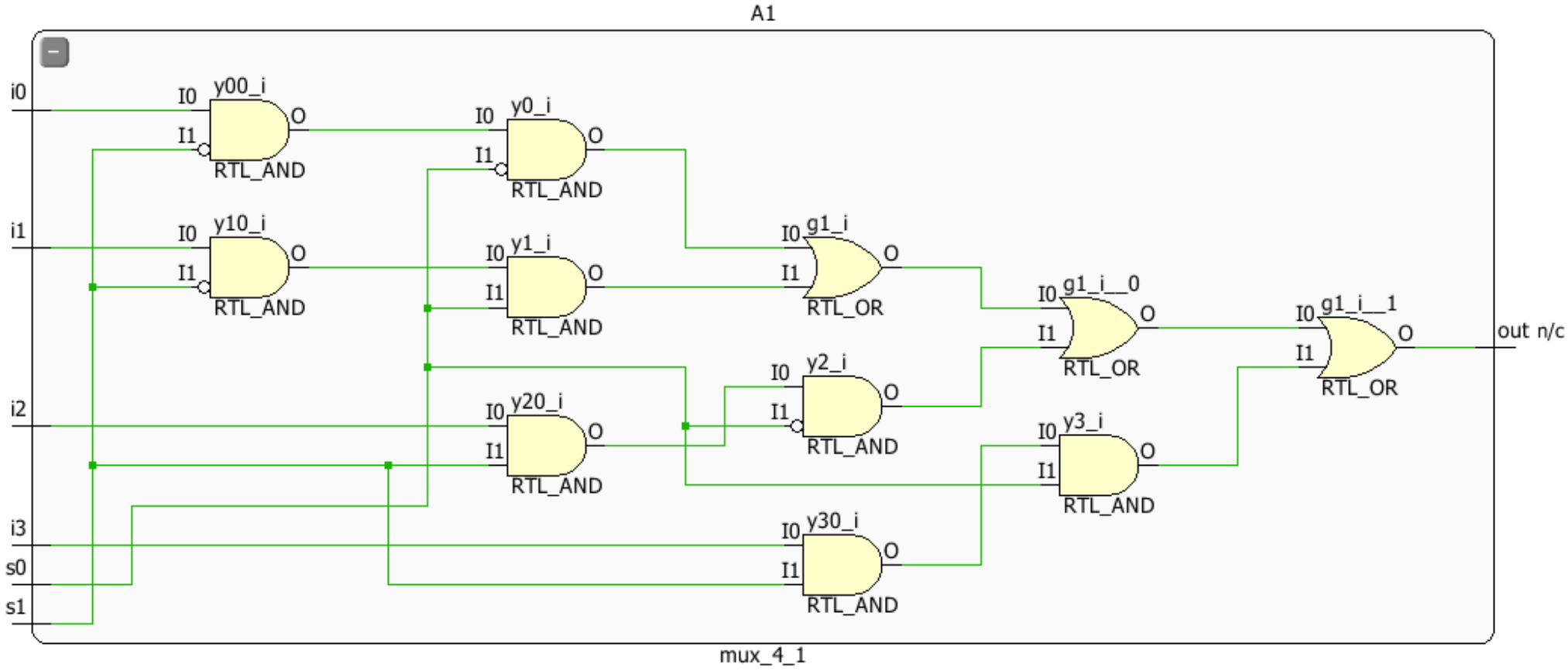
Structural modelling (gate level): 4:1 Multiplexer

```
module mux_4_to_1(output out, input
i0,i1,i2,i3,s1,s0 );
wire s1n, s0n; //internal wire declarations
wire y0, y1, y2, y3;
not(s1n,s1);
not(s0n,s0);
and(y0,i0,s1n,s0n); //3-input and gates instantiated
and(y1,i1,s1n,s0);
and(y2,i2,s1,s0n);
and(y3,i3,s1,s0);
or(out,y0,y1,y2,y3); //4-input or gates instantiated
endmodule
```

```
module tb;
wire out;
reg i0,i1,i2,i3,s1,s0;
mux_4_to_1 A1(out,i0,i1,i2,i3,s1,s0);
initial
begin
s1=0;s0=0;i0=0;i1=1;i2=1;i3=1;
#10 i0=1;i1=0;i2=0;i3=0;
#15 s0=1;i0=1;i1=0;i2=1;i3=1;
#5 i0=0;i1=1;i2=0;i3=0;
#10 s0=0;s1=1;i0=1;i1=1;i2=0;i3=1;
#5 i0=0;i1=0;i2=1;i3=0;
#10 s0=1;i0=1;i1=1;i2=1;i3=0;
#5 i0=0;i1=0;i2=0;i3=1;
#40 $finish;
end
endmodule
```

Simulation results:

Schematic


A1
i0
i1
i2
i3
s0
s1
y00_i
y10_i
y0_i
y1_i
y20_i
y30_i
y2_i
y3_i
g1_i
g1_i__0
g1_i__1
I0
I1
O
RTL_AND
RTL_OR
out n/c
mux_4_1

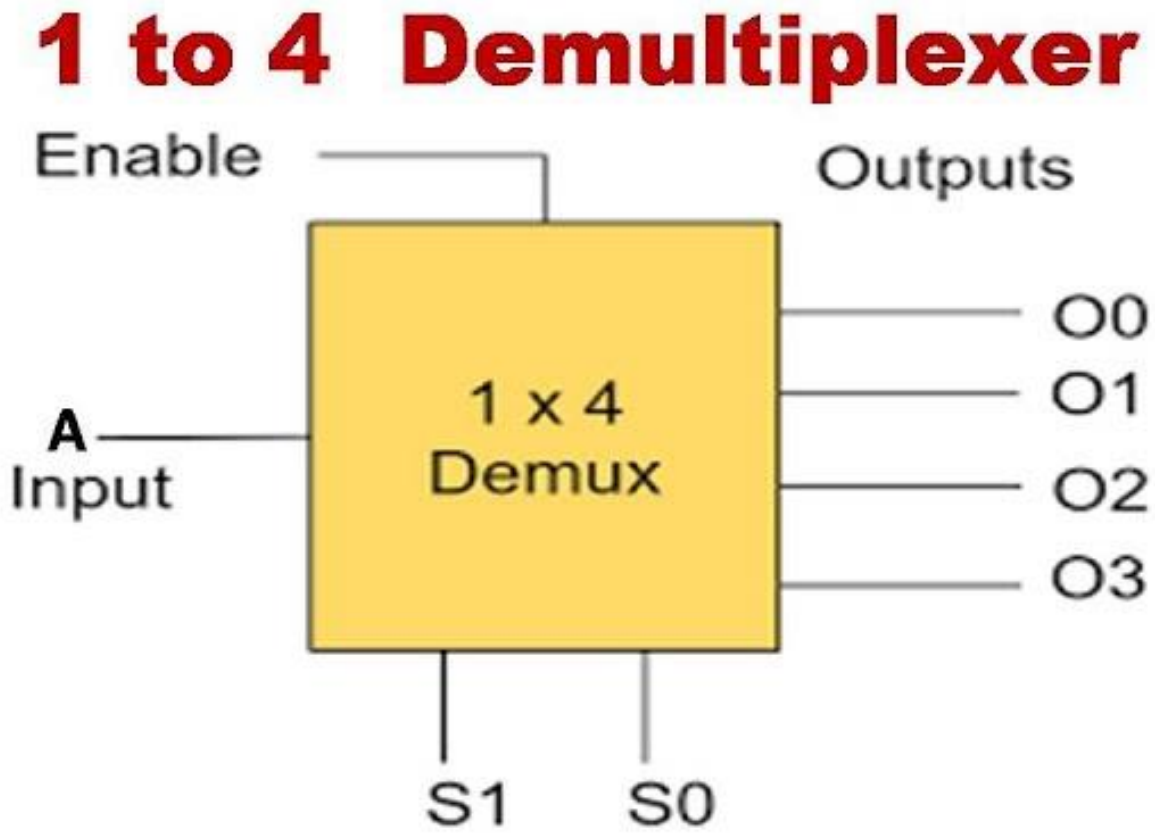
Data flow description modelling: **4-to-1 Multiplexer**

```
module mux_4_to_1( output out, input i0,i1,i2,i3,s1,s0);

Assign y = ((~S[1])&(~s[0])&I[0])|((~S[1])&(S[0])&I[1])|((S[1])&(~S[0])&I[2])|((S[0])&(S[1])&I[3]);

endmodule

//TEST BENCH REMAINS THE SAME
```

Behavioral modelling: **4-to-1 Multiplexer**

```
module mux_4_to_1( output reg out, input i0,i1,i2,i3,s1,s0 );
always @(i0,i1,i2,i3,s1,s0)
case ({s1,s0}) //{} concatenation
2'b00:out=i0;
2'b01:out=i1;
2'b10:out=i2;
2'b11:out=i3;
default:out=1'b0;
endcase
endmodule
//TEST BENCH REMAINS THE SAME
```

2. Gate-level modelling: Multiplexers and **Demultiplexers**

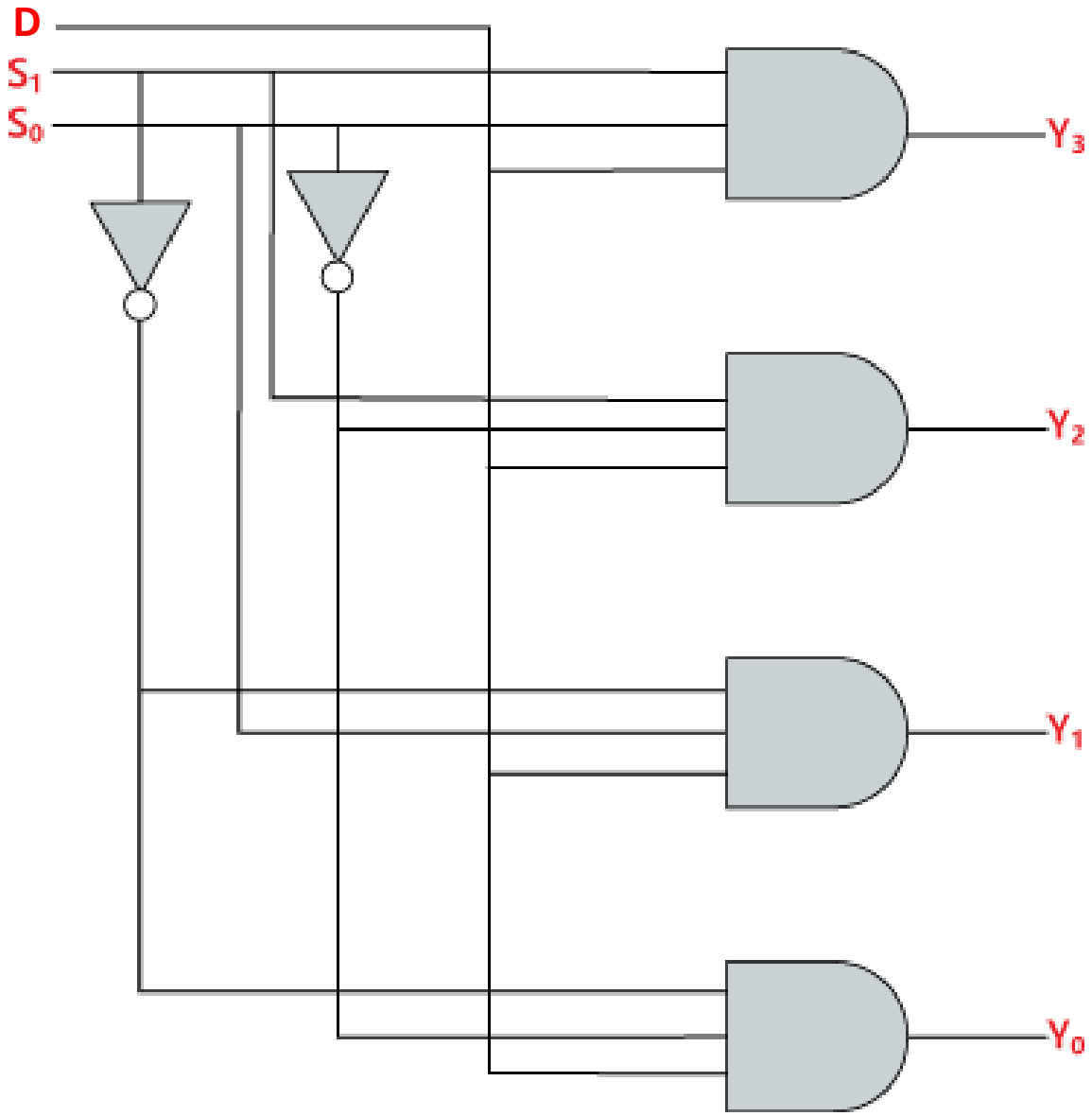


2.b Gate-level modelling: 1:4 **Demultiplexer**

S1	S0	D	Y3	Y2	Y1	Y0
0	0	0	0	0	0	**0**
0	0	1	0	0	0	**1**
0	1	0	0	0	**0**	0
0	1	1	0	0	**1**	0
1	0	0	0	**0**	0	0
1	0	1	0	**1**	0	0
1	1	0	**0**	0	0	0
1	1	1	**1**	0	0	0

$Y0 = \overline{S1}\ \overline{S0}\ D$ $\quad$ $Y1 = \overline{S1}\ S0\ D$ $\quad$ $Y2 = S1\ \overline{S0}\ D$ $\quad$ $Y3 = S1\ S0\ D$

2.b Gate-level modelling: 1:4 Demultiplexer



1:4 Demux: Structural modelling (gate level)

```
module demux(
input i,s1,s0,output o0,o1,o2,o3);
not(s1n,s1);
not(s0n,s0);
and(o0,s1n,s0n,i);
and(o1,s1n,s0,i);
and(o2,s1,s0n,i);
and(o3,s1,s0,i);
or(out,o0,o1,o2,o3);
endmodule
```

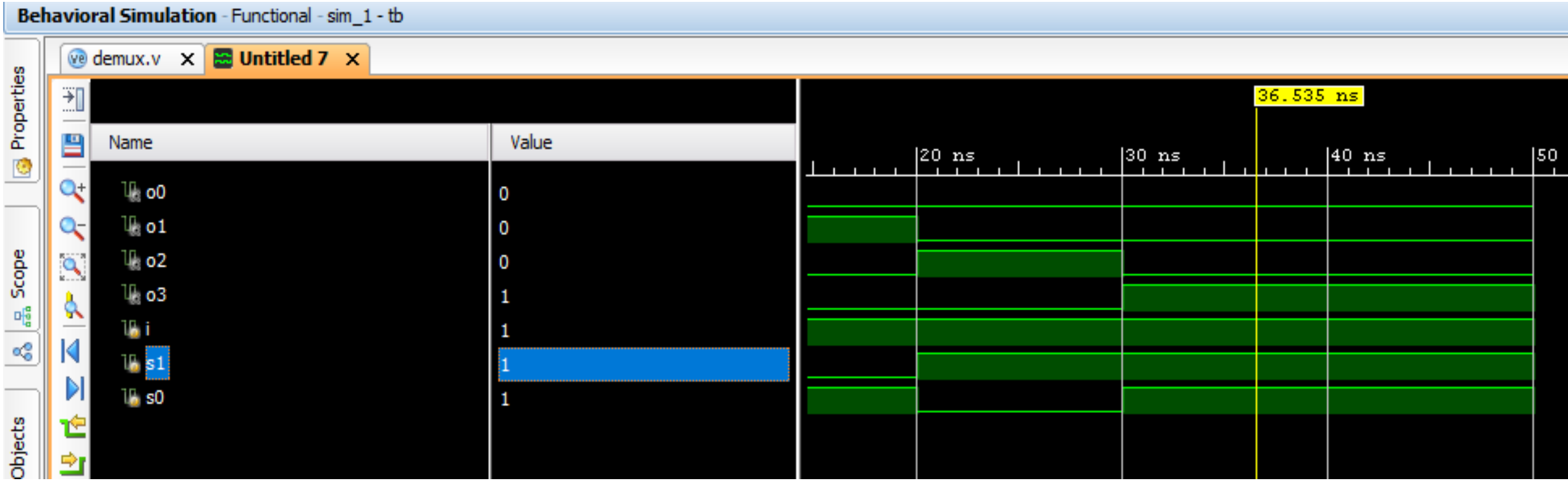
```
module tb;
wire o0,o1,o2,o3;
reg i,s1,s0;

demux A1(i,s1,s0,o0,o1,o2,o3);
initial
begin
s1=0;s0=0;i=1;
#10 s1=0;s0=1;i=1;
#10 s1=1;s0=0;i=1;
#10 s1=1;s0=1;i=1;
#10 $finish;
end
endmodule
```

1:4 Demux: Behavioural modelling

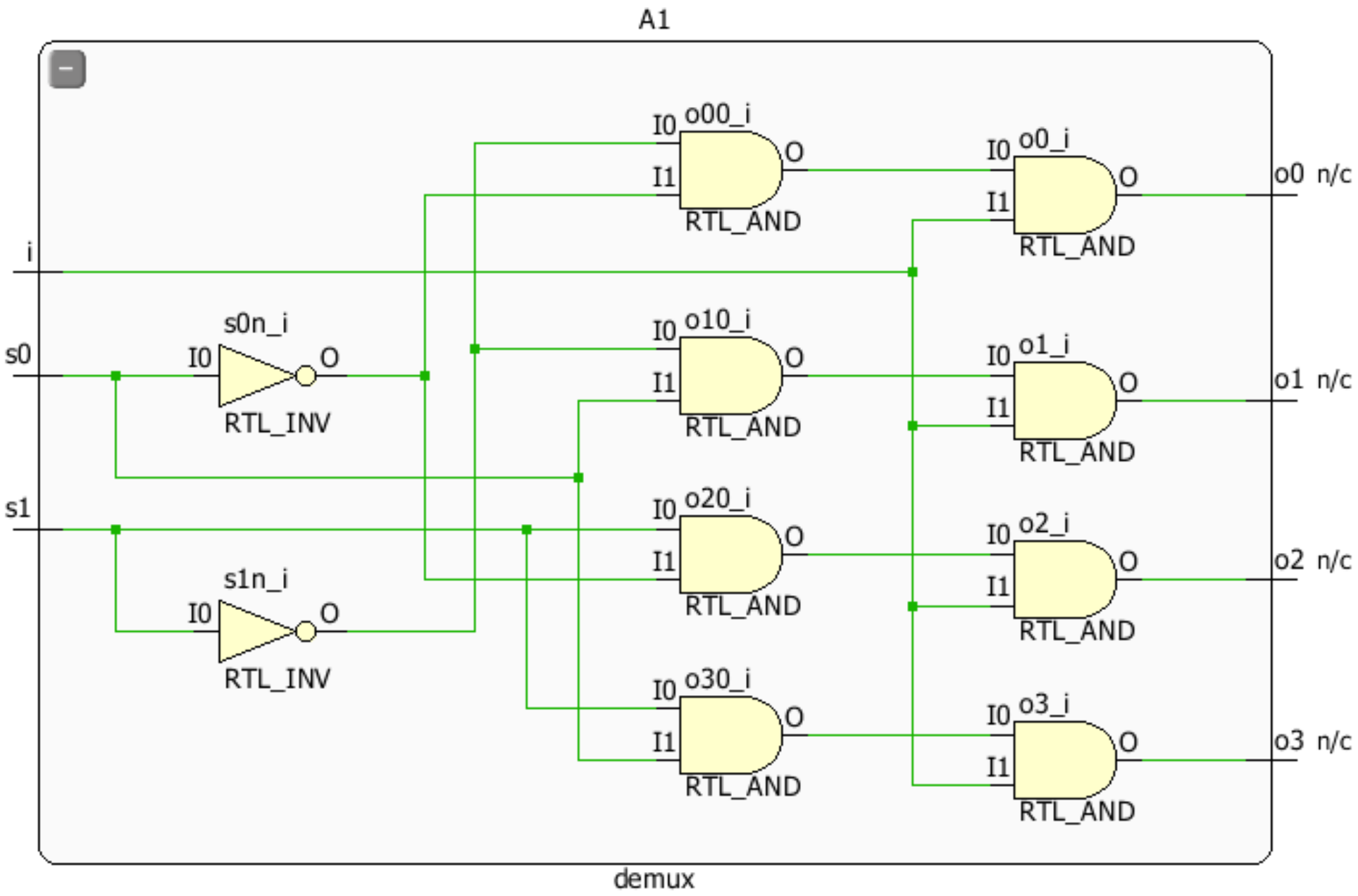
```
module demux( input i,s1,s0,
output o0,o1,o2,o3 );
always @(i,s1,s0)
case ({s1,s0}) //{} concatenation
2'b00:o0=i;
2'b01:o1=i;
2'b10:o2=i;
2'b11:o3=i;
default:o0=1'b0;
endcase
endmodule
```

```
module tb;
wire o0,o1,o2,o3;
reg i,s1,s0;

demux A1(i,s1,s0,o0,o1,o2,o3);
initial
begin
s1=0;s0=0;i=1;
#10 s1=0;s0=1;i=1;
#10 s1=1;s0=0;i=1;
#10 s1=1;s0=1;i=1;
#10 $finish;
end
endmodule
```

Simulation results:



Schematic:

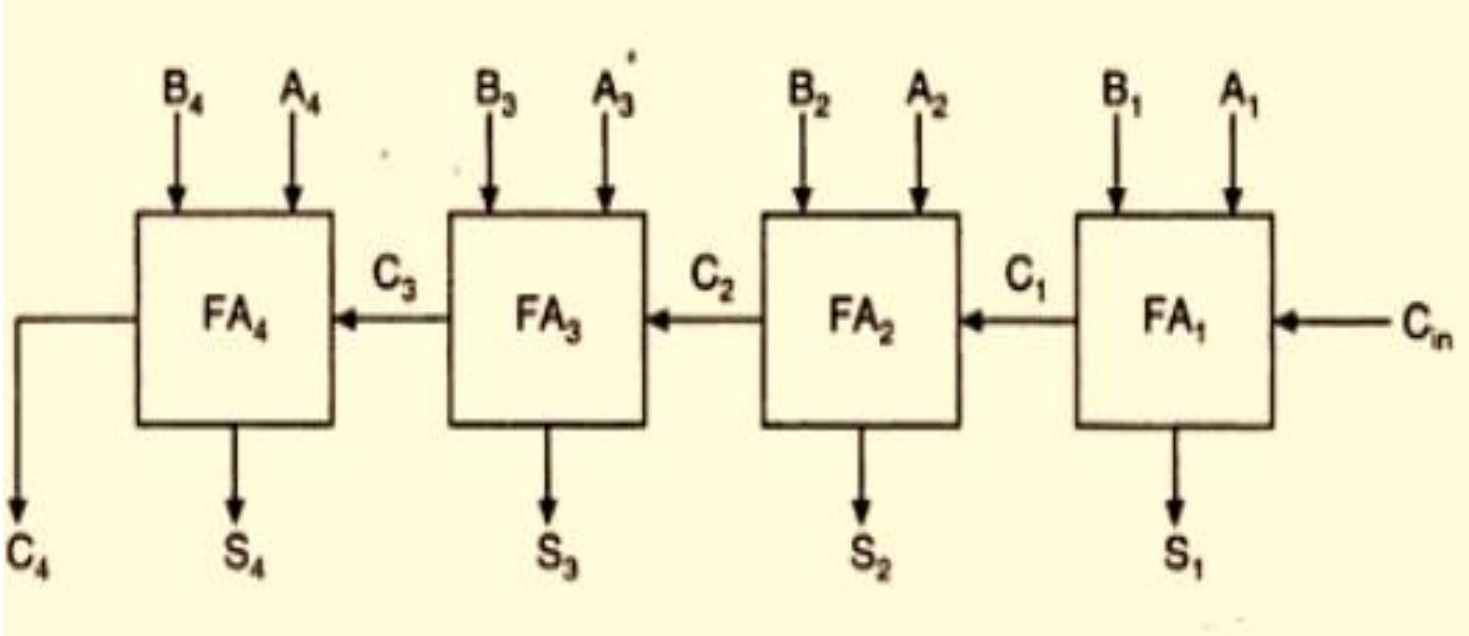

A1
i
s0
s1
s0n_i
I0
O
RTL_INV
s1n_i
I0
O
RTL_INV
o00_i
I0
I1
O
RTL_AND
o10_i
I0
I1
O
RTL_AND
o20_i
I0
I1
O
RTL_AND
o30_i
I0
I1
O
RTL_AND
o0_i
I0
I1
O
RTL_AND
o1_i
I0
I1
O
RTL_AND
o2_i
I0
I1
O
RTL_AND
o3_i
I0
I1
O
RTL_AND
o0 n/c
o1 n/c
o2 n/c
o3 n/c
demux

3. Ripple Carry adder

- **“Ripple Carry Adder”** - It’s a unique type of logic circuit used for adding the N-bit numbers in digital operations.
- A structure of multiple full adders is cascaded in a manner to gives the results of the addition of an n bit binary sequence.
- This adder includes cascaded full adders in its structure so, the carry will be generated at every full adder stage in a ripple-carry adder circuit.
- These carry output at each full adder stage is forwarded to its next full adder and there applied as a carry input to it.
- This process continues up to its last full adder stage.
- So, each carry output bit is rippled to the next stage of a full adder.
- By this reason, it is named as **“RIPPLE CARRY ADDER”**.

3. 4-bit Ripple Carry Adder

- In this adder, four full adders are connected in cascade.
- Co is the carry input bit and it is zero always.
- When this input carry ‘Co’ is applied to the two input sequences A1 A2 A3 A4 and B1 B2 B3 B4 then output represented with S1 S2 S3 S4 and output carry C4.

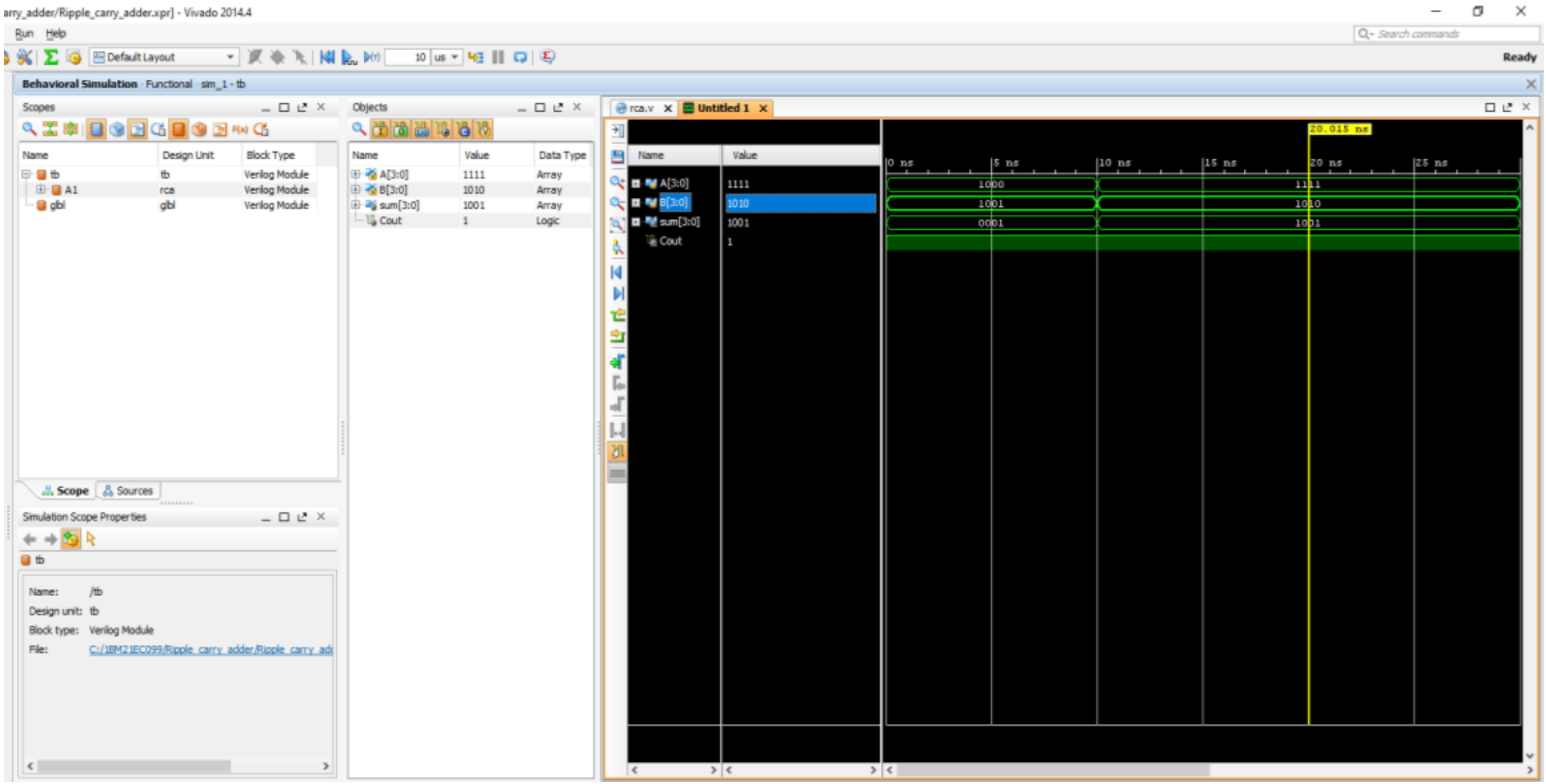


3. Data flow modelling: Ripple Carry adder

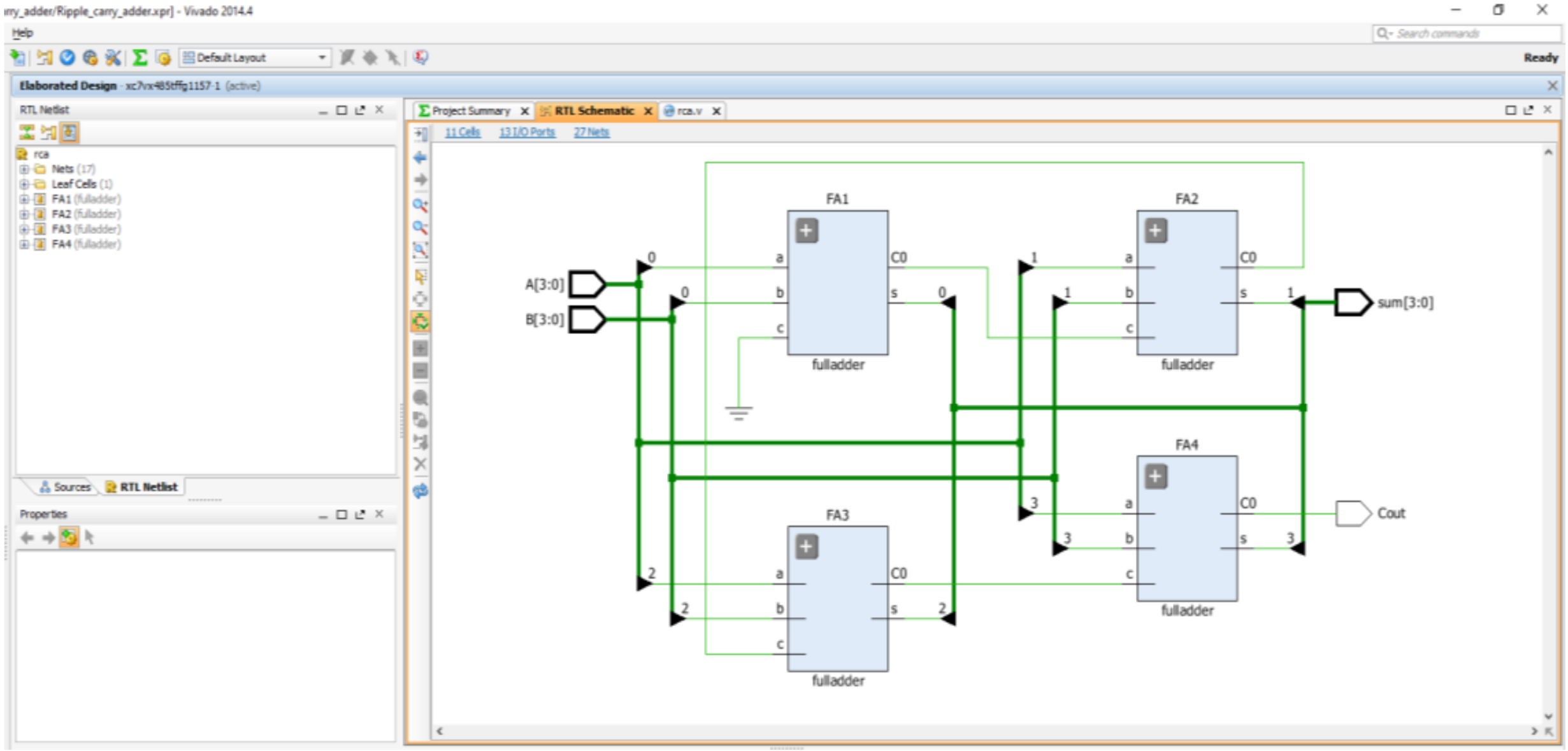
```
module rca(
input [3:0]A,B,
output [3:0]sum,
output Cout);
wire [3:0]C;
assign C[0]=0;
fulladder FA1(A[0],B[0],C[0],sum[0],C[1]);
fulladder FA2(A[1],B[1],C[1],sum[1],C[2]);
fulladder FA3(A[2],B[2],C[2],sum[2],C[3]);
fulladder FA4(A[3],B[3],C[3],sum[3],Cout);
endmodule
```

```
module tb;
reg [3:0]A,B;
wire [3:0]sum;
wire Cout;
rca A1(A,B,sum,Cout);
initial
begin
A=4'd8;B=4'd9;
#10 A=4'hF;B=4'hA;
#20 $finish;
end
endmodule

module fulladder(input a,b,c,output s,C0);
xor(s,a,b,c);
and(y1,a,b);
and(y2,b,c);
and(y3,c,a);
or(C0,y1,y2,y3);
endmodule
```

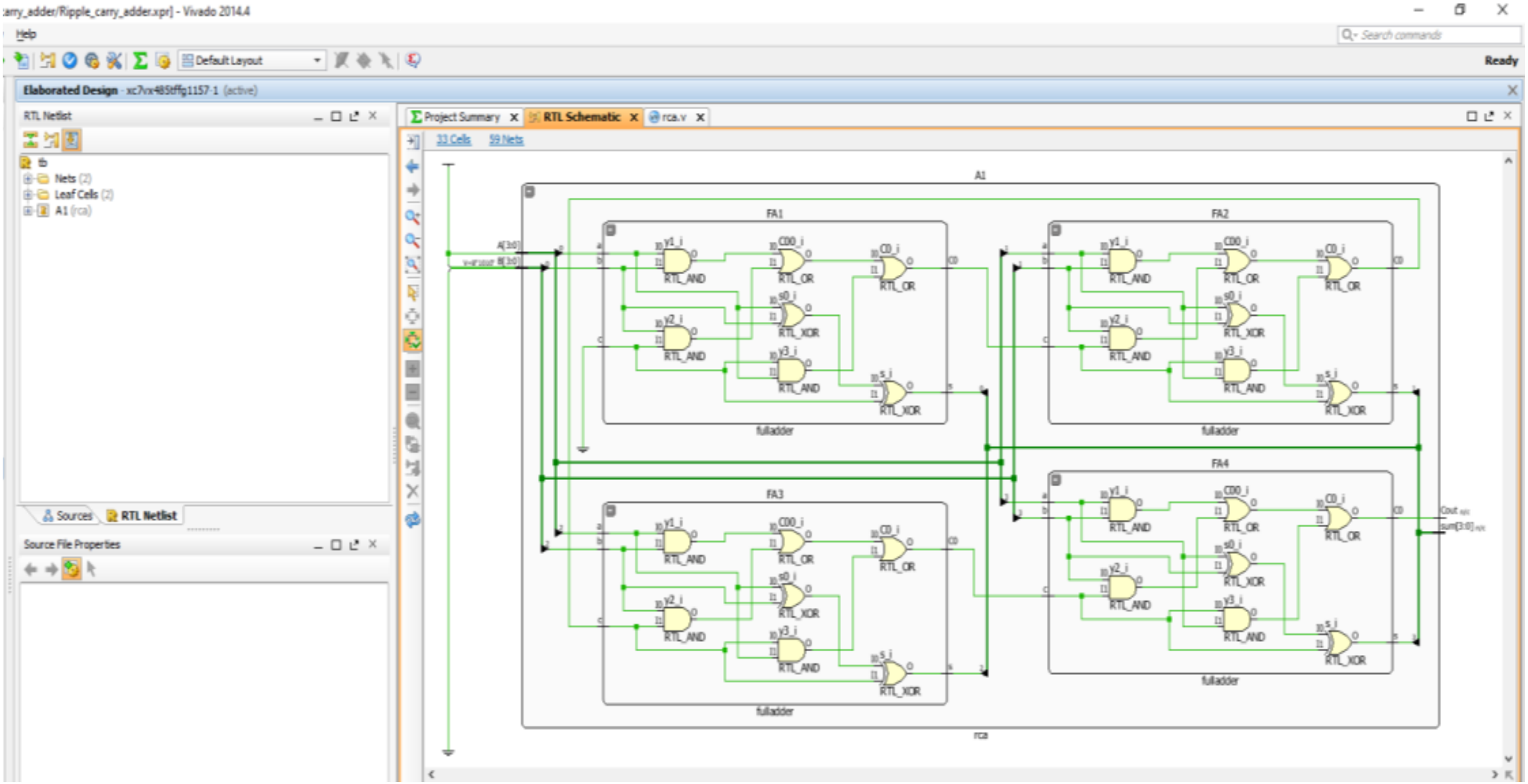
Simulation


arry_adder/Ripple_carry_adder.xpr] - Vivado 2014.4
Run Help
Default Layout
Ready
Behavioral Simulation - Functional - sim_1 - tb
Scopes
Name Design Unit Block Type
tb tb Verilog Module
A1 rca Verilog Module
glbl glbl Verilog Module
Scope Sources
Simulation Scope Properties
tb
Name: /tb
Design unit: tb
Block type: Verilog Module
File: C:/1BM21EC099/Ripple_carry_adder/Ripple_carry_ad
Objects
Name Value Data Type
A[3:0] 1111 Array
B[3:0] 1010 Array
sum[3:0] 1001 Array
Cout 1 Logic
rca.v
Untitled 1
20.015 ns
0 ns 5 ns 10 ns 15 ns 20 ns 25 ns
1000 1111
1001 1010
0001 1001

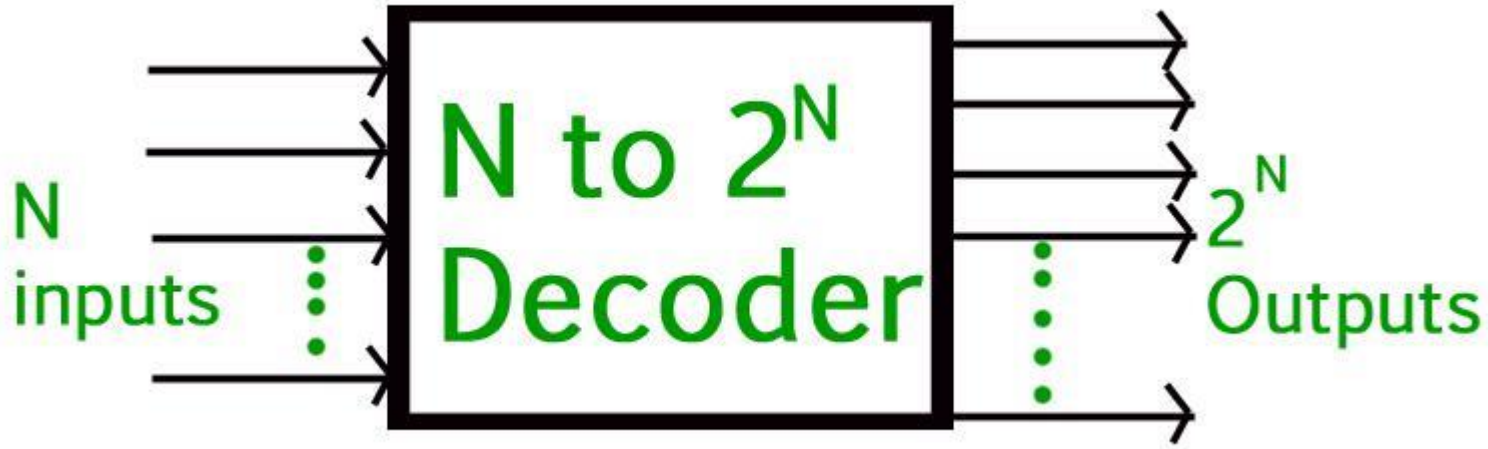
Schematic


ry_adder/Ripple_carry_adder.xpr] - Vivado 2014.4
Help
Default Layout
Ready
Elaborated Design - xc7vx485tffg1157-1 (active)
RTL Netlist
rca
Nets (17)
Leaf Cells (1)
FA1 (fulladder)
FA2 (fulladder)
FA3 (fulladder)
FA4 (fulladder)
Sources
RTL Netlist
Properties
Project Summary
RTL Schematic
rca.v
11 Cells
13 I/O Ports
27 Nets
A[3:0]
B[3:0]
FA1
FA2
FA3
FA4
fulladder
a
b
c
C0
s
sum[3:0]
Cout

Schematic


carry_adder/Ripple_carry_adder.xpr] - Vivado 2014.4
Help
Default Layout
Ready
Elaborated Design - xc7vx485tffg1157-1 (active)
RTL Netlist
tb
Nets (2)
Leaf Cells (2)
A1 (rca)
Sources
RTL Netlist
Source File Properties
Project Summary
RTL Schematic
rca.v
33 Cells
59 Nets
A1
FA1
FA2
FA3
FA4
fulladder
rca
RTL_AND
RTL_OR
RTL_XOR

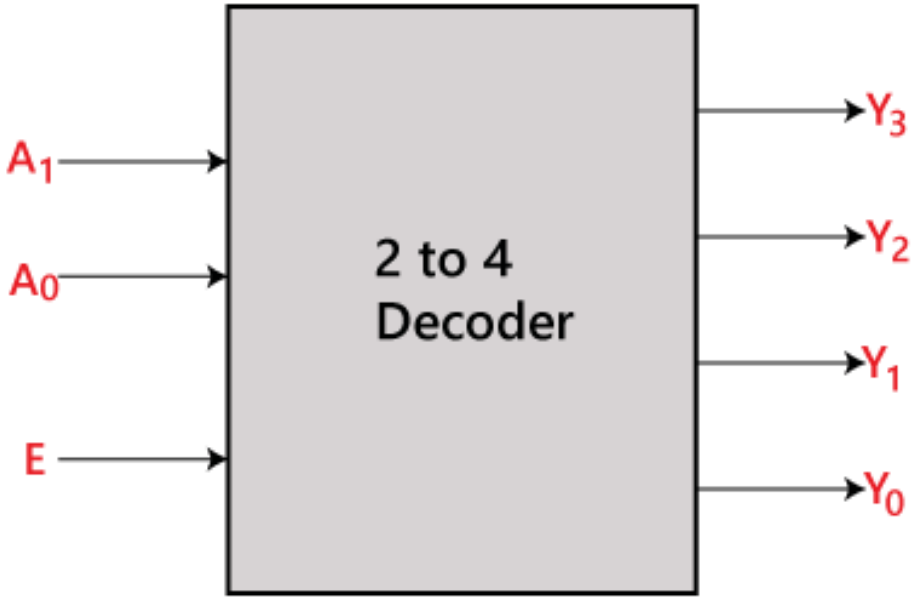
3. Decoder



- Decoder
- This Decoder is a combinational logic circuit, and its purpose is to decode the data given to it. It is made of n number of input lines and 2^n output lines. For every probable input condition, there are various output signals and depending on the input.
- Need of Decoder: Memory Addressing, Seven-Segment Displays, Control Systems

3. Decoder - 2 to 4 line decoder:

- In the 2 to 4 line decoder, there is a total of three inputs, i.e., A0, and A1 and E and four outputs, i.e., Y0, Y1, Y2, and Y3.
- For each combination of inputs, when the enable 'E' is set to 1, one of these four outputs will be 1.



Truth Table:

Enable	INPUTS		OUTPUTS			
E	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	X	X	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

3. Decoder - 2 to 4 line decoder:

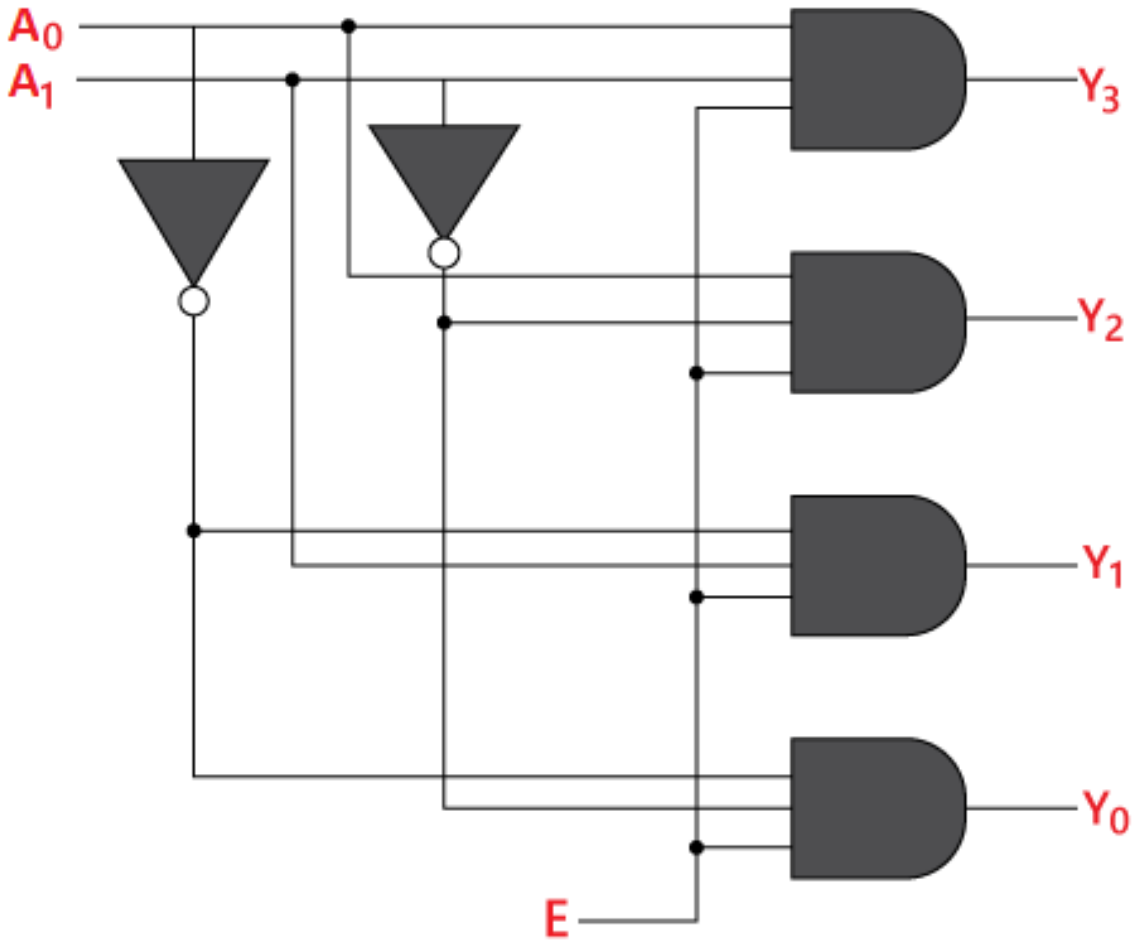
- The logical expression and circuit of the term Y0, Y0, Y2, and Y3 is as follows:

$Y_3 = E.A_1.A_0$

$Y_2 = E.A_1.A_0'$

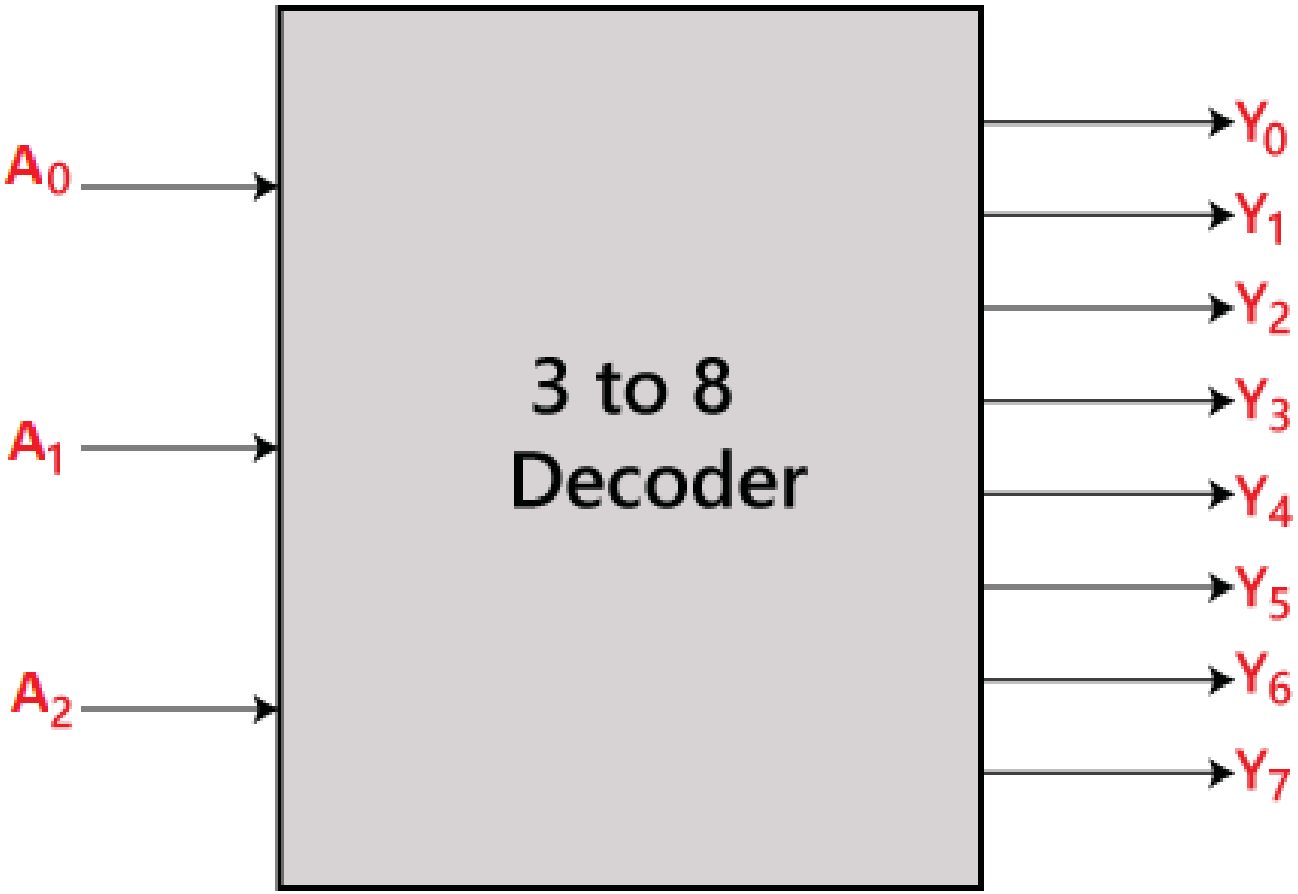
$Y_1 = E.A_1'.A_0$

$Y0 = E.A_1'.A_0'$



3. Decoder - **3 to 8 line decoder:**

- The 3 to 8 line decoder is also known as **Binary to Octal Decoder.**
- In a 3 to 8 line decoder, there is a total of **8 outputs**, i.e., **Y0, Y1, Y2, Y3, Y4, Y5, Y6, and Y7** and **3 inputs, i.e., A0, A1, and A2**.
- This circuit has an enable input 'E'.
- Just like 2 to 4 line decoder, when enable 'E' is set to 1, one of these 8 outputs will be 1.
- The block diagram and the truth table of the 3 to 8 line encoder are given below.



3. Decoder - **3 to 8 line decoder:**

Truth Table:

Enable	INPUTS			Outputs							
E	A_2	A_1	A_0	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	x	x	x	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	1
1	0	0	1	0	0	0	0	0	0	1	0
1	0	1	0	0	0	0	0	0	1	0	0
1	0	1	1	0	0	0	0	1	0	0	0
1	1	0	0	0	0	0	1	0	0	0	0
1	1	0	1	0	0	1	0	0	0	0	0
1	1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	1	0	0	0	0	0	0	0

The logical expression of the term Y_0, Y_1, Y_2, Y_3, Y_4, Y_5, Y_6, and Y_7 is as follows:

$$Y_0 = A_0'.A_1'.A_2'$$

$$Y_1 = A_0.A_1'.A_2'$$

$$Y_2 = A_0'.A_1.A_2'$$

$$Y_3 = A_0.A_1.A_2'$$

$$Y_4 = A_0'.A_1'.A_2$$

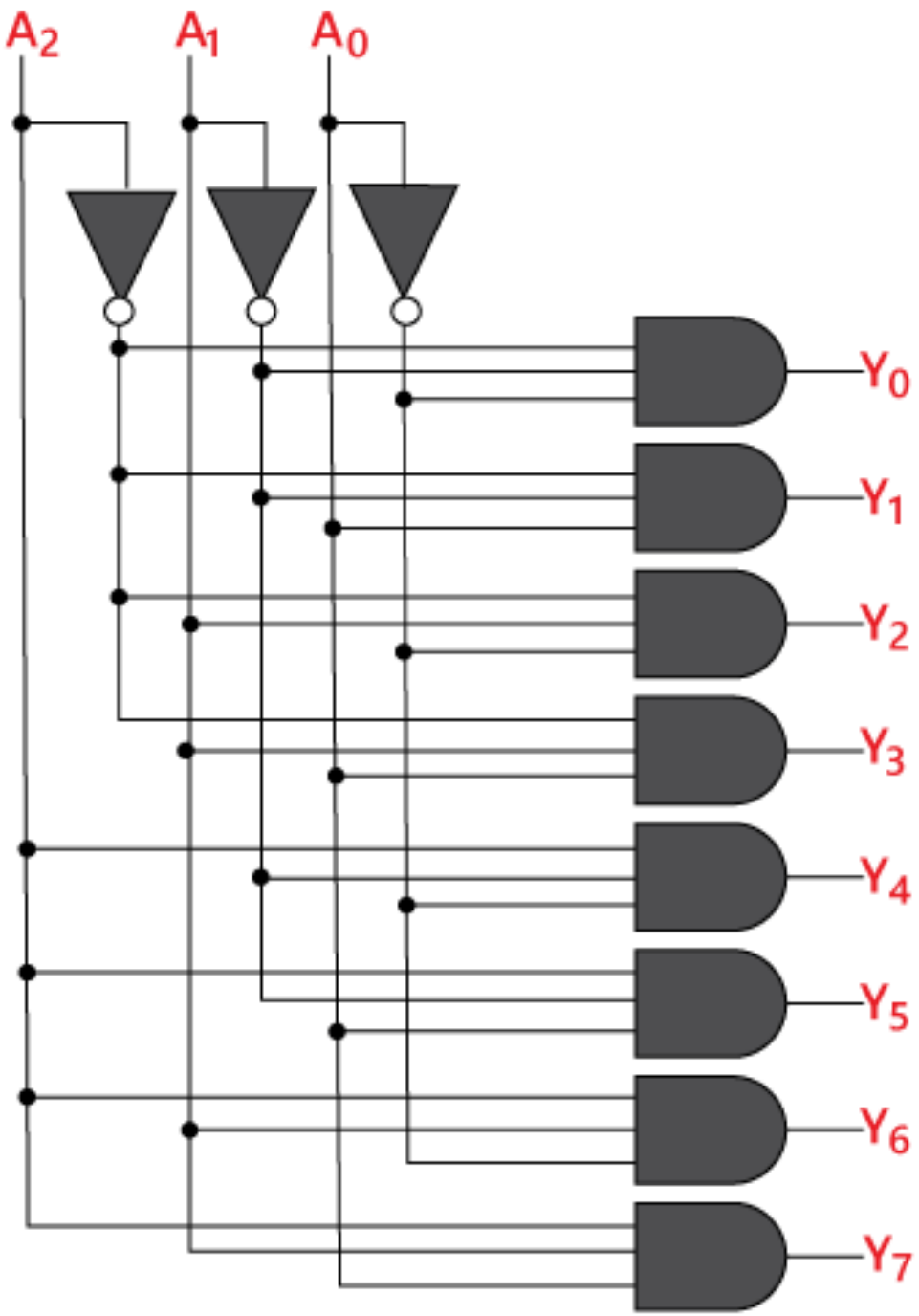
$$Y_5 = A_0.A_1'.A_2$$

$$Y_6 = A_0'.A_1.A_2$$

$$Y_7 = A_0.A_1.A_2$$

3. Decoder –
3 to 8 line decoder:

- Logical circuit of the above expressions is given below:



a) Structural modelling (**gate level**): Decoder

```
module decoder( input E,a2,a1,a0, output
[7:0]Y );
not(a2n,a2);
not(a1n,a1);
not(a0n,a0);
and(Y[0],a2n,a1n,a0n,E);
and(Y[1],a2n,a1n,a0,E);
and(Y[2],a2n,a1,a0n,E);
and(Y[3],a2n,a1,a0,E);
and(Y[4],a2,a1n,a0n,E);
and(Y[5],a2,a1n,a0,E);
and(Y[6],a2,a1,a0n,E);
and(Y[7],a2,a1,a0,E);
endmodule
```

```
module tb;
wire [7:0]Y;
reg E,a2,a1,a0;
decoder A1(E,a2,a1,a0,Y);
initial begin
E=1;a2=0;a1=0;a0=0;
#5 E=1;a2=0;a1=0;a0=1;
#5 E=1;a2=0;a1=1;a0=0;
#5 E=1;a2=0;a1=1;a0=1;
#5 E=1;a2=1;a1=0;a0=0;
#5 E=1;a2=1;a1=0;a0=1;
#5 E=1;a2=1;a1=1;a0=0;
#5 E=1;a2=1;a1=1;a0=1;
#15 $finish;
end
endmodule
```

b) Data Flow modelling: Decoder

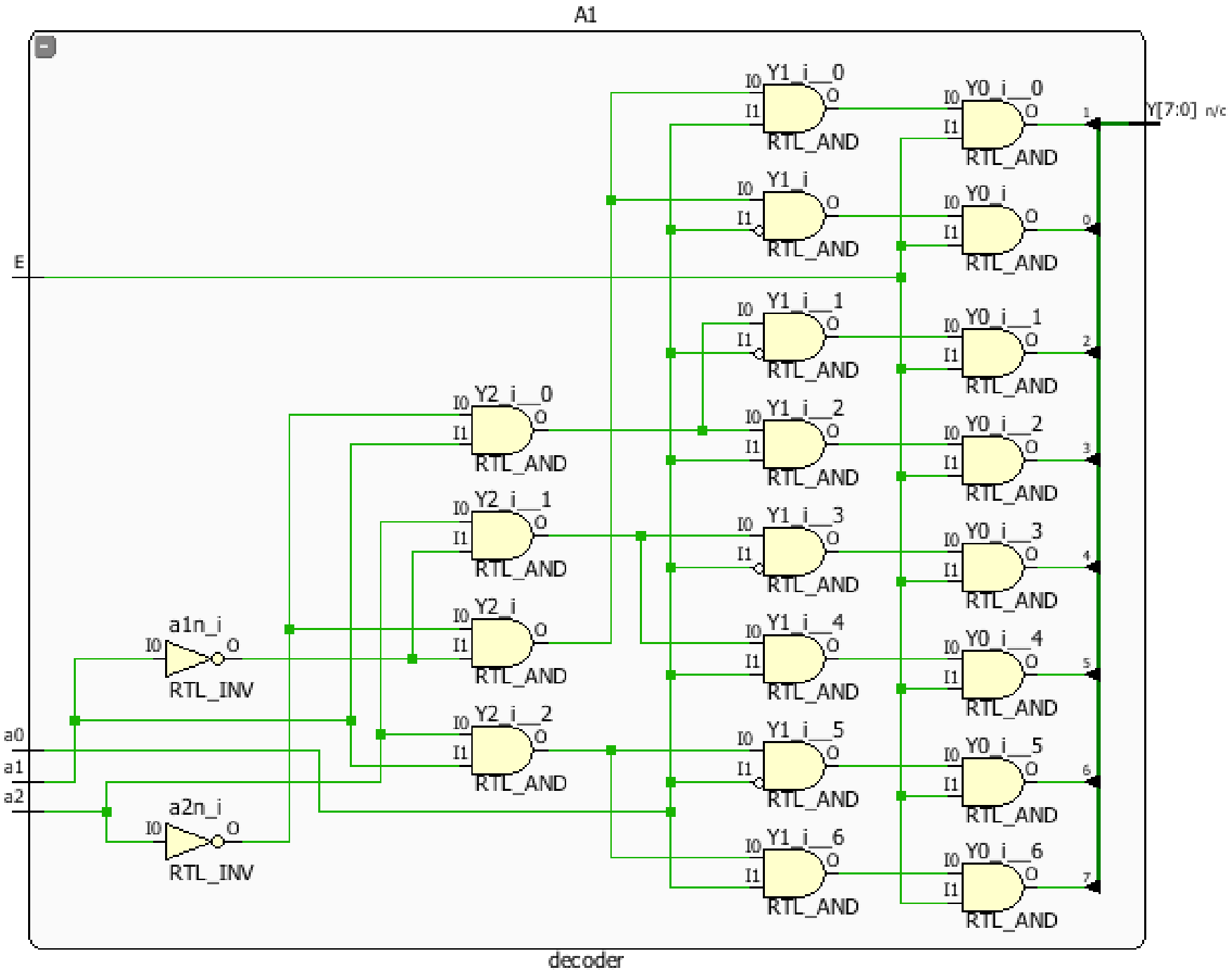
```
module decoder( input E,a2,a1,a0, output [7:0]Y);
assign Y[0]=((~a2)&(~a1)&(~a0)&E);
assign Y[1]=((~a2)&(~a1)&(a0)&E);
assign Y[2]=((~a2)&(a1)&(~a0)&E);
assign Y[3]=((~a2)&(a1)&(a0)&E);
assign Y[4]=((a2)&(~a1)&(~a0)&E);
assign Y[5]=((a2)&(~a1)&(a0)&E);
assign Y[6]=((a2)&(a1)&(~a0)&E);
assign Y[7]=((a2)&(a1)&(a0)&E);
endmodule
```

c) Behavioral Modelling: Decoder

```
module decoder( input E,a2,a1,a0, output reg [7:0]Y);
always@(E,a2,a1,a0)
if(E==1)
case({a2,a1,a0})
3'b000:Y=8'd1;
3'b001:Y=8'd2;
3'b010:Y=8'd4;
3'b011:Y=8'd8;
3'b100:Y=8'd16;
3'b101:Y=8'd32;
3'b110:Y=8'd64;
3'b111:Y=8'd128;
default:Y=1'b0;
endcase
endmodule
```

Simulation Output

Schematic


A1
E
a0
a1
a2
a1n_i
a2n_i
RTL_INV
Y2_i__0
Y2_i__1
Y2_i
Y2_i__2
Y1_i__0
Y1_i
Y1_i__1
Y1_i__2
Y1_i__3
Y1_i__4
Y1_i__5
Y1_i__6
Y0_i__0
Y0_i
Y0_i__1
Y0_i__2
Y0_i__3
Y0_i__4
Y0_i__5
Y0_i__6
RTL_AND
I0
I1
O
Y[7:0]
n/c
decoder